Helm Chart Quality Checker

Course: CSE3253 DevOps [PE6]

University: Manipal University Jaipur

An automated quality scanner for Kubernetes Helm chart YAML files

Problem Statement

Kubernetes Standard

Industry standard for deploying applications at scale using Helm charts (YAML files)

Configuration Risks

Single misconfiguration can cause security breaches, crashes, or resource exhaustion

No Automation

No automated way to check chart quality before deployment

Our Solution: Upload Helm chart → get instant quality score, grade, and list of issues with fixes

System Architecture

5 services running together via Docker Compose

Frontend

React + Vite

Backend

Node.js + Express

MongoDB

Users & reports

Linter

Python + FastAPI

Redis

Job queue & cache

Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite, TailwindCSS	User interface
Backend	Node.js, Express	REST API
Linter	Python, FastAPI, Uvicorn	Quality checks
Database	MongoDB, Mongoose	Persistent storage
Queue	Redis, BullMQ	Async job processing
Auth	JWT, bcrypt	User authentication
DevOps	Docker, Docker Compose	Containerization
CI/CD	GitHub Actions	Automated pipeline

How It Works

01

User Login & Upload

User logs in and uploads Helm chart .yaml file

02

Cache Check

Backend checks Redis cache — returns instantly if same file scanned before

03

Queue Job

If not cached, job added to BullMQ queue with jobId

04

Poll Status

Frontend polls /api/scan/status/:jobId every 2 seconds

05

Process Job

BullMQ worker picks up job, sends file to Python linter

06

Run Checks

Linter parses YAML, runs 8 checks, calculates score

07

Store Result

Result saved to MongoDB, cached in Redis for 1 hour

08

Display Results

Frontend displays score, grade, and all issues with fixes

The 8 Quality Checks

#	Check	Severity	Deduction
1	RUN_AS_ROOT	🔴 HIGH	−20
2	PRIVILEGED_CONTAINER	🔴 HIGH	−20
3	HOST_NETWORK	🔴 HIGH	−20
4	NO_RESOURCE_LIMITS	🟡 MEDIUM	−10
5	NO_LIVENESS_PROBE	🟡 MEDIUM	−10
6	NO_READINESS_PROBE	🟡 MEDIUM	−10
7	LATEST_IMAGE_TAG	🔵 LOW	−5
8	NO_RESOURCE_REQUESTS	🔵 LOW	−5

Score starts at 100, deductions applied per issue found. Maximum possible deduction = 100 points.

Scoring & Grading

90–100: A
Production ready

75–89: B
Minor issues

60–74: C
Needs improvement

40–59: D
Serious issues

0–39: F
Not production ready

Bad Chart Example

3 HIGH + 2 MEDIUM + 1 LOW = −75
Score: 25 → Grade: F

Good Chart Example

1 LOW = −5
Score: 95 → Grade: A

Authentication & Security

Password Hashing

bcrypt with 10 salt rounds — never stored as plain text

JWT Tokens

Issued on login with 7 day expiry, verified on every request

Route Protection

authMiddleware verifies token on protected routes, returns 401 Forbidden if unauthorized

Secret Management

Kubernetes Secrets store JWT and DB credentials — never in code

Async Processing & Caching

BullMQ + Redis

Why not call linter directly?

- Linting could be slow for large files
- Direct call would block HTTP request
- No way to track progress

How BullMQ solves this:

- Upload returns jobId immediately
- Job sits in Redis queue
- Worker processes in background
- Frontend polls for status
- Failed jobs automatically retried

Redis Caching

Same file scanned twice? Second result is instant.

- First scan: result stored in Redis with 1 hour TTL
- Second scan: Redis cache hit, returned immediately
- Key format: scan:<fileHash>
- Saves processing time and reduces linter load

DevOps Implementation

Docker & Kubernetes

Each service has Dockerfile, 5 services in docker-compose.yml, full Kubernetes manifests in infrastructure/kubernetes/

CI/CD Pipeline

GitHub Actions at pipelines/.github/workflows/ci-cd.yml, triggers on every git push, builds, tests, lints, deploys

Project Structure

src/main (frontend, backend, linter), infrastructure (docker, kubernetes), pipelines (CI/CD), src/test, sample-charts

Key Achievement: The project itself follows every best practice it checks — our own Kubernetes manifests pass all 8 checks with Grade A